Data Track

Interview Preparation Series • Professional Edition

Databases

08

Compiled with LaTeX • June 30, 2026

harsha.somisetty01@gmail.com

In This Edition

1 Overview --- What It Is 3

2 Why It Exists 3

3 Why FAANG Cares (Company-Specific) 3

4 Core Concepts 4

- 4.1 Relational Model 4
- 4.2 Schema 4
- 4.3 CAP Theorem Applied to Databases 4
- 4.4 Normalization vs Denormalization 4

5 SQL Deep Dive 5

- 5.1 Joins --- Complete Reference 5
- 5.2 CTEs (Common Table Expressions) 7
- 5.3 Window Functions 8
- 5.4 Query Execution Plan / EXPLAIN 9

6 Indexing & Internals 10

- 6.1 B-Tree vs B+Tree vs LSM-Tree 10
- 6.2 Clustered vs Non-Clustered Index 11
- 6.3 How Index Speeds Lookups 12

7 ACID, Isolation Levels & Locking 12

- 7.1 ACID Properties 13
- 7.2 THE Isolation Level × Anomaly Matrix 13
- 7.3 MVCC (Multi-Version Concurrency Control) 14
- 7.4 Optimistic vs Pessimistic Locking 15
- 7.5 Deadlocks in Databases 15
- 7.6 Lock Types 15

8 Architecture / Diagrams 16

- 8.1 B+Tree Structure (Full ASCII) 16
- 8.2 LSM-Tree Compaction Flow 16
- 8.3 Replication Topology 17
- 8.4 Sharding / Partitioning 17

9 NoSQL Deep Dive 18

- 9.1 When to Pick Which Database 18
- 9.2 NoSQL Families Comparison 18
- 9.3 Cassandra Architecture 18
- 9.4 DynamoDB 19
- 9.5 MongoDB 19

10 Real-World Examples 19

11 Real-Life Analogies 19

12 Memory Tricks / Mnemonics 20

- 12.1 ACID 21
- 12.2 Isolation Levels (weakest to strongest) 21
- 12.3 Joins Memory Aid 21
- 12.4 B+Tree vs LSM Choice 21
- 12.5 Window Function Rank Types 21
- 12.6 NoSQL Family by Access Pattern 21
- 12.7 CAP Theorem 21

13 Common Interview Questions 21

- 13.1 SQL Query Questions (Frequently Asked) 22
- 13.2 System-Level DB Questions 22
- 13.3 Follow-Up Questions Interviewers Love 23

14 Senior-Level Discussion Points 23

- 14.1 Index Trade-offs 24
- 14.2 Write vs Read Optimization 24
- 14.3 Hot Partitions 24

15 Typical Mistakes Candidates Make **25**

16 How This Connects To Other Topics **25**

17 FAANG Interview Tips **26**

18 Revision Cheat Sheet **26**

18.1 10-Minute Summary . . . 26
18.2 Key Numbers to Know . . . 27
18.3 Most Important Concepts for Interviews . . . 27
18.4 Cheat-Sheet Summary Table . . . 27

Audience: FAANG-level deep mastery. First-principles explanations, crisp mnemonics, heavy interview focus.

1 Overview --- What It Is

A **database** is a structured, persistent store of data with controlled access, concurrency management, and durability guarantees.

Category	Definition	Examples
Relational (SQL)	Tables, rows, columns; schema-on-write; ACID by default	PostgreSQL, MySQL, Oracle, SQLite, SQL Server
Key-Value	Hash map at scale; fastest reads/writes	Redis, DynamoDB (simple use), Memcached
Document	JSON/BSON documents, flexible schema	MongoDB, Couchbase, Firestore
Column-Family	Wide rows, optimized for large analytical scans	Cassandra, HBase, Bigtable
Graph	Nodes + edges, relationship traversal	Neo4j, Amazon Neptune
Time-Series	Optimized for timestamp-indexed sequential writes	InfluxDB, TimescaleDB
NewSQL	SQL semantics + distributed scalability	Google Spanner, CockroachDB, TiDB

2 Why It Exists

Core problem: Applications need data to outlive the process that creates it. Without a database:

- Data lives only in RAM → lost on crash
- Concurrent writers corrupt each other's changes
- No way to search, filter, or aggregate efficiently

Databases solve four fundamental problems:

1. **Persistence** — Durability after crash/power loss
2. **Concurrency** — Multiple readers/writers simultaneously without corruption
3. **Query efficiency** — Find 1 row in 1 billion without scanning everything
4. **Integrity** — Constraints (FK, UNIQUE, NOT NULL) prevent bad data

3 Why FAANG Cares (Company-Specific)

Compa	Their DB Tech	Why They Care Deeply
Amazon	DynamoDB, Aurora, RDS	DynamoDB is their crown jewel. Every Amazonian must understand KV stores, partitioning, eventual consistency. Aurora reimagines log-structured storage.
Google	Spanner, Bigtable, Firestore	Spanner achieves global ACID via TrueTime. Bigtable invented the column-family model. Expect Spanner internals questions.

Compa	Their DB Tech	Why They Care Deeply
Meta	MySQL (at insane scale), RocksDB, TAO	TAO is a graph KV store for social graph. RocksDB (LSM-tree) underpins dozens of systems. MySQL sharding at 10B+ users.
Micros	Azure SQL, Cosmos DB	Cosmos DB offers 5 tunable consistency levels. Azure SQL is SQL Server cloud-native.
Netflix	Cassandra, EVCache (Memcached)	Cassandra for multi-region, always-available write patterns. EVCache for session/token caching.
Uber	MySQL → Schemaless → CockroachDB	Uber's MySQL sharding war stories are famous. Schemaless is a custom document store on top of MySQL.

Interview signal: Knowing *which company uses which DB and why* shows systems thinking, not just textbook knowledge.

4 Core Concepts

4.1 Relational Model

Data organized in **relations (tables)**. A table is a set of tuples (rows). Each column has a declared type. **Primary key** uniquely identifies a row. **Foreign key** references another table's PK, enforcing referential integrity.

4.2 Schema

- **Schema-on-write (SQL):** Schema defined upfront; data must conform at insert time. Safer, slower to evolve.
- **Schema-on-read (NoSQL):** No schema enforcement at write; application interprets at read. More flexible, more dangerous.

4.3 CAP Theorem Applied to Databases

Under a **network partition**, choose two of three:

- **Consistency** — All reads see the latest write
- **Availability** — Every request gets a response
- **Partition Tolerance** — System works despite dropped messages

```
CAP Choices in Practice:
  CP (Consistent + Partition-tolerant): HBase, Zookeeper, Spanner
  AP (Available + Partition-tolerant): Cassandra, DynamoDB, CouchDB
  CA (Consistent + Available): Traditional single-node RDBMS (not truly distributed)
```

PACELC extends CAP: even when no partition, there's a latency vs consistency trade-off.

4.4 Normalization vs Denormalization

Aspect	Normalization	Denormalization
Goal	Eliminate data redundancy	Optimize read performance
Storage	Less (no duplication)	More (intentional duplication)
Write cost	Lower (update one place)	Higher (update multiple places)
Read cost	Higher (need joins)	Lower (data pre-joined)

Aspect	Normalization	Denormalization
Anomalies	Prevented (update/delete/insert)	Possible
Use when	OLTP, data correctness critical	OLAP, denormalized reports, caching

Normal Forms (quick):

- **1NF:** Atomic values, no repeating groups
- **2NF:** 1NF + no partial dependencies (every non-key col depends on full PK)
- **3NF:** 2NF + no transitive dependencies (non-key col doesn't depend on another non-key col)
- **BCNF:** 3NF + every determinant is a candidate key

Interview takeaway: Most OLTP systems target 3NF. Most data warehouses intentionally denormalize (star/snowflake schema).

5 SQL Deep Dive

5.1 Joins --- Complete Reference

```
Tables:
  employees:  id | name     | dept_id
              1  | Alice    | 10
              2  | Bob      | 20
              3  | Charlie  | 30  ← dept doesn't exist

  departments: id | dept_name
               10 | Engineering
               20 | Marketing
               40 | HR         ← no employee
```

INNER JOIN --- Only matching rows

```sql
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;
```

```
Result:
  name   | dept_name
  Alice  | Engineering
  Bob    | Marketing

  [Charlie excluded — dept 30 not in departments]
  [HR excluded — no employee in dept 40]
```

```
Venn diagram:
  employees ●●●●●●●●  departments
              [████]
          ↑ only overlap
```

LEFT JOIN --- All left rows + matching right (NULLs if no match)

```sql
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;
```

```
Result:
  name    | dept_name
  Alice   | Engineering
  Bob     | Marketing
  Charlie | NULL          ← Charlie kept, dept is NULL
```

```
Venn diagram:
  [████████████████]  departments
  employees
  ↑ all of left side
```

RIGHT JOIN --- All right rows + matching left

```sql
-- All departments, even those without employees
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.id;
```

```
Result:
  name  | dept_name
  Alice | Engineering
  Bob   | Marketing
  NULL  | HR          ← HR kept, employee is NULL
```

FULL OUTER JOIN --- Union of LEFT + RIGHT

```sql
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.id;
```

```
Result:
  name    | dept_name
  Alice   | Engineering
  Bob     | Marketing
  Charlie | NULL
  NULL    | HR
```

CROSS JOIN --- Cartesian product (M × N rows)

```sql
SELECT e.name, d.dept_name
FROM employees e CROSS JOIN departments d;
-- 3 employees × 3 departments = 9 rows
```

SELF JOIN --- Table joined with itself

```sql
-- Find employees with the same department
SELECT a.name, b.name, a.dept_id
FROM employees a
JOIN employees b ON a.dept_id = b.dept_id AND a.id < b.id;
```

Anti-Join pattern (find rows with no match)

```sql
-- Employees with no department
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;
-- Result: Charlie
```

5.2 CTEs (Common Table Expressions)

Purpose: Name a subquery so it can be referenced (and reused) within a larger query. Makes complex queries readable. Can be **recursive** for tree/graph traversal.

Non-Recursive CTE Example

```sql
-- Find departments where average salary > company average
WITH dept_avg AS (
    SELECT dept_id,
           AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept_id
),
company_avg AS (
    SELECT AVG(salary) AS company_avg_sal
    FROM employees
)
SELECT d.dept_name, da.avg_sal, ca.company_avg_sal
FROM dept_avg da
JOIN departments d ON da.dept_id = d.id
CROSS JOIN company_avg ca
WHERE da.avg_sal > ca.company_avg_sal;
```

```
Output (hypothetical):
  dept_name    | avg_sal | company_avg_sal
  Engineering  | 120000  | 95000
```

Recursive CTE Example --- Org chart traversal

```sql
WITH RECURSIVE org_chart AS (
    -- Base case: find the CEO (no manager)
    SELECT id, name, manager_id, 1 AS depth
    FROM employees
    WHERE manager_id IS NULL
```

```sql

    UNION ALL

    -- Recursive case: find direct reports
    SELECT e.id, e.name, e.manager_id, oc.depth + 1
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT name, depth
FROM org_chart
ORDER BY depth;
```

```
Output:
  name    | depth
  CEO     | 1
  CTO     | 2
  VP Eng  | 3
  Alice   | 4
```

Interview takeaway: Recursive CTEs are the standard SQL answer for hierarchical/tree data. Know the base case + recursive case pattern.

5.3 Window Functions

Key insight: Window functions compute across a "window" of rows related to the current row, **without collapsing rows** like GROUP BY does.

```sql
SELECT
    name,
    dept_id,
    salary,
    RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank,
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rank,
    ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
    LAG(salary)  OVER (PARTITION BY dept_id ORDER BY salary DESC) AS prev_salary,
    LEAD(salary) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS next_salary,
    SUM(salary)  OVER (PARTITION BY dept_id)                      AS dept_total,
    AVG(salary)  OVER (PARTITION BY dept_id)                      AS dept_avg,
    SUM(salary)  OVER (ORDER BY salary ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
    ↪ running_total
FROM employees;
```

Sample Data:

```
  name    | dept_id | salary
  Alice   | 10      | 120000
  Bob     | 10      | 90000
  Charlie | 10      | 90000
  Dave    | 20      | 80000
```

Output:

```
  name    | dept | salary | dept_rank | dense_rank | row_num | prev_salary | next_salary | dept_total | dept_avg  | running_total
  Alice   | 10   | 120000 | 1         | 1          | 1       | NULL        | 90000       | 300000     | 100000    | 120000
  Bob     | 10   | 90000  | 2         | 2          | 2       | 120000      | 90000       | 300000     | 100000    | 210000
  Charlie | 10   | 90000  | 2         | 2          | 3       | 90000       | NULL        | 300000     | 100000    | 300000
```

```
Dave    | 20  | 80000  | 1        | 1         | 1      | NULL      | NULL      | 80000     | 80000    | 380000
```

RANK vs DENSE_RANK vs ROW_NUMBER:

- `RANK()`: gaps after ties (1, 2, 2, **4**)
- `DENSE_RANK()`: no gaps (1, 2, 2, **3**)
- `ROW_NUMBER()`: always unique (1, 2, 3, 4)

Common window function interview patterns:

```sql
-- Top N per group (most asked!)
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn <= 3;

-- Running totals
SELECT date, revenue,
       SUM(revenue) OVER (ORDER BY date) AS cumulative_revenue
FROM daily_sales;

-- Moving average (last 7 days)
SELECT date, revenue,
       AVG(revenue) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM daily_sales;

-- Percent of total
SELECT name, salary,
       ROUND(100.0 * salary / SUM(salary) OVER (), 2) AS pct_of_total
FROM employees;
```

5.4 Query Execution Plan / EXPLAIN

```sql
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;
```

Key nodes in a query plan:

Node Type	Meaning	Cost
Seq Scan	Full table scan — reads every row	O(n), slow for large tables
Index Scan	Traverse B+tree, fetch rows by pointer	O(log n + k) for k results
Index Only Scan	Answer from index alone, no heap fetch	Fastest — ”covering index”
Bitmap Heap Scan	Collect row pointers first, then fetch	Good for range queries
Nested Loop	For each row in outer, scan inner	O(n*m) worst case
Hash Join	Build hash of smaller table, probe it	O(n+m), good for large
Merge Join	Both tables sorted, merge	O(n log n + m log m)

What to look for in EXPLAIN:

1. **Seq Scan on large table** → missing index
2. **High rows estimate vs actual** → stale statistics → run ANALYZE

3. **Nested Loop with large outer** → wrong join order, no index on inner
4. **Sort on large result** → consider index to avoid sort
5. **cost=X..Y** → startup cost .. total cost (planner estimates)

6 Indexing & Internals

6.1 B-Tree vs B+Tree vs LSM-Tree

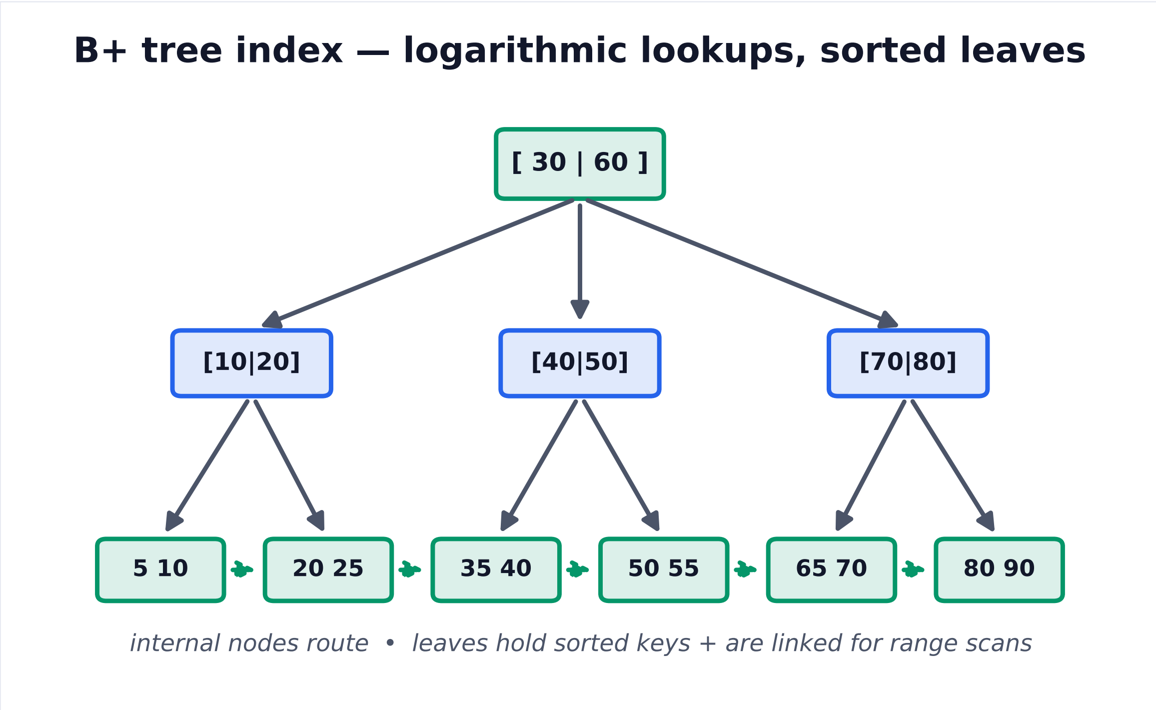

Figure 1. B+ trees keep data sorted with a high branching factor, so lookups touch only O(log n) nodes (few disk pages). Linked leaves make range queries cheap.

B+Tree (Used in PostgreSQL, MySQL InnoDB, SQL Server)

```
Structure: All data in LEAF nodes only. Internal nodes = routing keys only.
Leaf nodes linked as a doubly-linked list → efficient range scans.

                    [30 | 70]
                   /    |    \
           [10|20]  [40|60]  [80|90]
           /  |  \    |  |     |   \
          10  20  30 40 60  80   90
          ←──────────────────────→  (leaf linked list)
```

Properties:

- Height typically 3-4 for millions of rows (branching factor ~100-1000)
- All lookups: O(log n) with small constant
- Range scans: traverse leaf linked list — very cache-friendly
- Reads and writes both O(log n)
- **In-place updates** — pages modified in place

Why not B-Tree (non-plus)?

- B-Trees store data in internal nodes too

- Range scans require traversal through all levels
- B+Trees are strictly better for databases

LSM-Tree (Log-Structured Merge-Tree) --- Used in Cassandra, RocksDB, LevelDB, HBase

```
Write path:
  Write → WAL → MemTable (in-memory sorted) → SSTable (immutable sorted file on disk)

  MemTable: [  sorted key-value pairs in memory  ]
      ↓ (when full, flush)
  L0:   [SSTable1] [SSTable2] [SSTable3]  (may overlap)
      ↓ (compaction)
  L1:   [SSTable — sorted, no overlap]
      ↓ (compaction)
  L2:   [SSTable — sorted, no overlap, larger]
```

Compaction:

```
  [file A: a=1, c=3, e=5]   [file B: b=2, c=4, d=6]
          ↓  merge sort
  [merged: a=1, b=2, c=4, d=6, e=5]   ← c=4 wins (newer)
```

Properties:

- **Writes:** Sequential only → very fast (SSD-optimized, no random writes)
- **Reads:** May need to check multiple SSTables → use Bloom filters to skip files
- **Space amplification:** Old versions accumulate until compaction
- **Write amplification:** Data written multiple times during compaction

Comparison Table

Aspect	B+Tree	LSM-Tree
Write performance	Moderate (random I/O)	High (sequential I/O)
Read performance	High (direct lookup)	Moderate (multi-file check)
Range scans	Excellent (leaf list)	Good (sorted SSTables)
Space usage	Efficient	Higher (until compaction)
Write amplification	Low	High (compaction rewrites)
Read amplification	Low (one path)	Higher (check multiple files)
Best for	Read-heavy, OLTP	Write-heavy, time-series, logging
Used by	PostgreSQL, MySQL, SQLite	Cassandra, RocksDB, LevelDB, HBase

6.2 Clustered vs Non-Clustered Index

Clustered Index

The actual table rows are stored in index order. Only one per table.

```
Clustered Index (InnoDB Primary Key):
  B+Tree leaf nodes ARE the actual row data

  Leaf: [PK=1 | name=Alice | salary=120000 | ...]
        [PK=2 | name=Bob   | salary=90000  | ...]
        [PK=3 | name=...   | ...           | ...]
        ← physically ordered by PK on disk
```

Implications:

- PK lookups are fastest possible (no additional fetch)

- Range scans by PK are very efficient (sequential disk I/O)
- Inserts in PK order are fast; random PK order causes **page splits**
- **UUID as PK is bad for InnoDB** — random inserts cause constant page splits and fragmentation

Non-Clustered (Secondary) Index

Index is separate structure; leaf nodes contain key + pointer to actual row.

```
Secondary Index on (salary):

  B+Tree leaf: [salary=90000 → row pointer (PK=2)]
               [salary=120000 → row pointer (PK=1)]

  To fetch full row:
    1. Traverse secondary index → get PK
    2. Traverse clustered index with PK → get full row
    This second lookup = "bookmark lookup" or "key lookup"
```

Covering Index: Index includes all columns the query needs — no heap fetch required.

```sql
-- Query:
SELECT name, salary FROM employees WHERE dept_id = 10;

-- Covering index: (dept_id, name, salary)
-- Index leaf contains: [dept_id=10 | name=Alice | salary=120000]
-- No need to touch the actual table rows!
```

Interview takeaway: Secondary index lookup = 2 B+Tree traversals (secondary → clustered). Covering index eliminates the second traversal. This is why you see "Index Only Scan" in EXPLAIN.

6.3 How Index Speeds Lookups

Without index on `salary`:

```
SELECT * FROM employees WHERE salary = 90000;
→ Full table scan: read all N rows, compare each
→ O(N) time, O(N) I/O
```

With B+Tree index on `salary`:

```
  1. Start at root of B+Tree
  2. Compare 90000 with internal node keys → go right/left
  3. Traverse ~log_100(N) levels (typically 3-4 for millions of rows)
  4. Arrive at leaf node → row pointer
  5. Fetch actual row by pointer
→ O(log N) time, O(log N) I/O
```

Index selectivity: High cardinality = high selectivity = better index.

- `salary` — thousands of distinct values → highly selective → great index
- `gender` — 2 distinct values → low selectivity → index often not used (full scan faster)

7 ACID, Isolation Levels & Locking

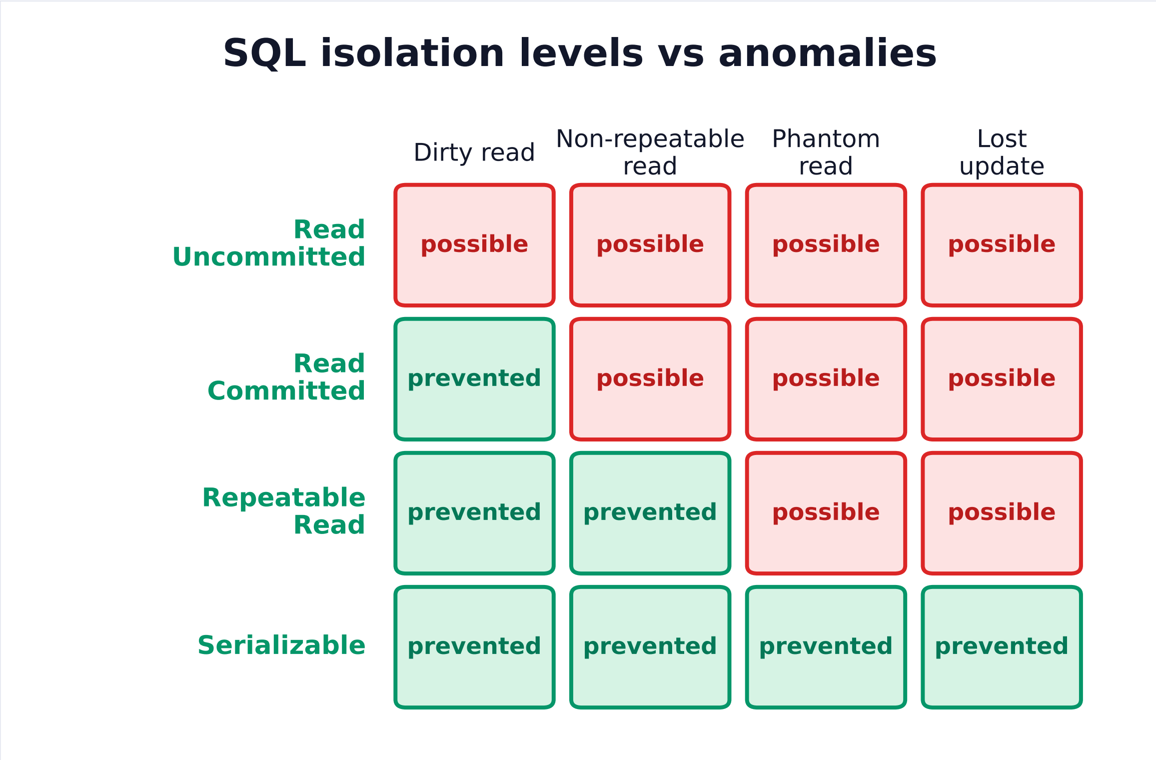

Figure 2. Stronger isolation prevents more anomalies but costs concurrency. Serializable is safest; most OLTP systems default to Read Committed and add locks where needed.

7.1 ACID Properties

Property	Definition	Implementation
Atomicity	Transaction is all-or-nothing	Rollback log (undo log) on failure
Consistency	DB goes from one valid state to another	Constraints, triggers, cascades enforced
Isolation	Concurrent transactions don't interfere	Locks, MVCC
Durability	Committed transactions survive crashes	WAL (Write-Ahead Log), fsync

WAL (Write-Ahead Log):

```
Before modifying any page on disk:
  1. Write the change to the WAL (sequential, fast)
  2. fsync the WAL
  3. Only then modify actual data pages
On crash: replay WAL to reconstruct committed state
```

7.2 THE Isolation Level × Anomaly Matrix

This is **the most-asked DB theory question in FAANG interviews.**

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Lost Update
Read Uncommitted	Possible	Possible	Possible	Possible
Read Committed	**Prevented**	Possible	Possible	Possible
Repeatable Read	**Prevented**	**Prevented**	Possible*	**Prevented**
Serializable	**Prevented**	**Prevented**	**Prevented**	**Prevented**

*MySQL InnoDB's Repeatable Read also prevents phantom reads via gap locks (stronger than standard SQL spec)

Anomaly Definitions

Dirty Read: Reading uncommitted data from another transaction.

```
T1: UPDATE balance = 500 (not committed yet)
T2: SELECT balance → sees 500  ← DIRTY READ
T1: ROLLBACK
T2 used data that never existed!
```

Non-Repeatable Read: Same query returns different results within same transaction.

```
T1: SELECT salary → 90000
T2: UPDATE salary = 100000; COMMIT
T1: SELECT salary → 100000  ← DIFFERENT! Non-repeatable read
```

Phantom Read: New rows appear in re-executed query.

```
T1: SELECT * WHERE salary > 80000 → 5 rows
T2: INSERT new employee with salary=100000; COMMIT
T1: SELECT * WHERE salary > 80000 → 6 rows  ← PHANTOM!
```

Lost Update: Two transactions both read a value, both update it; one update is lost.

```python
T1: read balance=1000, compute 1000+100=1100
T2: read balance=1000, compute 1000+200=1200, write 1200; COMMIT
T1: write 1100; COMMIT
Balance = 1100 — T2's +200 is LOST!
```

7.3 MVCC (Multi-Version Concurrency Control)

Key insight: Instead of locking rows for reads, keep **multiple versions** of each row. Readers see a consistent snapshot; writers create new versions without blocking readers.

```python
PostgreSQL MVCC (simplified):
  Row has: xmin (created by tx), xmax (deleted/updated by tx)

  INSERT (tx 100): row {xmin=100, xmax=NULL, data="Alice, 90000"}
  UPDATE (tx 200): old row {xmin=100, xmax=200, data="Alice, 90000"}
                   new row {xmin=200, xmax=NULL, data="Alice, 100000"}

  Tx 150 (started before tx 200): sees old row (xmin=100 < 150, xmax=200 > 150)
  Tx 250 (started after tx 200 commits): sees new row (xmin=200 < 250)
```

Benefits:

- Readers never block writers
- Writers never block readers
- Each transaction sees a consistent snapshot of the DB

Downside: Dead versions accumulate → PostgreSQL's **VACUUM** process cleans them up.

7.4 Optimistic vs Pessimistic Locking

Aspect	Pessimistic Locking	Optimistic Locking
Assumption	Conflicts are frequent	Conflicts are rare
Mechanism	Lock row at read time (SELECT FOR UPDATE)	Check version/timestamp at commit
Blocking	Yes — other txs wait	No — detect conflict at commit
Throughput	Lower (lock contention)	Higher (no waiting)
Best for	High-contention workloads, financial	Low-contention, read-heavy
Implementation	`SELECT ... FOR UPDATE`, `LOCK TABLE`	Version column, compare-and-swap

```sql
-- Pessimistic locking
BEGIN;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;  -- locks the row
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
COMMIT;

-- Optimistic locking (application-level)
SELECT balance, version FROM accounts WHERE id = 1;
-- version = 5, balance = 1000
-- ... compute new balance = 900
UPDATE accounts
SET balance = 900, version = 6
WHERE id = 1 AND version = 5;  -- only updates if version hasn't changed
-- If 0 rows updated → conflict! Retry.
```

7.5 Deadlocks in Databases

Deadlock: Transaction A waits for lock held by B; B waits for lock held by A. Circular wait.

```
T1: LOCK row 1 → waiting for row 2 lock (held by T2)
T2: LOCK row 2 → waiting for row 1 lock (held by T1)
    ← DEADLOCK!
```

Detection: DB maintains a wait-for graph. Cycle = deadlock. One transaction chosen as victim and rolled back.

Prevention strategies:

1. **Lock ordering:** Always acquire locks in the same order (e.g., lower ID first)
2. **Timeout:** Transaction aborts if it can't acquire lock within N ms
3. **Wound-Wait / Wait-Die:** Timestamp-based schemes (older tx wounds/kills newer)
4. **Deadlock detection + victim selection:** Most RDBMS default approach

Interview takeaway: Deadlock = circular dependency in the wait-for graph. DB detects and kills one victim. Application must retry on deadlock error.

7.6 Lock Types

Lock	Symbol	Compatibility	Usage
Shared (S)	Read lock	S+S compatible; S+X not	SELECT ... LOCK IN SHARE MODE

Lock	Symbol	Compatibility	Usage
Exclusive (X)	Write lock	X incompatible with all	UPDATE, DELETE, SELECT FOR UPDATE
Intention Shared (IS)	Table-level intent	Indicates row-level S lock	Auto-set before row S lock
Intention Exclusive (IX)	Table-level intent	Indicates row-level X lock	Auto-set before row X lock
Gap lock	Range lock	Prevents phantom inserts	InnoDB Repeatable Read
Next-key lock	Row + gap	= row lock + gap lock	InnoDB default in RR

8 Architecture / Diagrams

8.1 B+Tree Structure (Full ASCII)

```
                ┌───────────┐
                │ Root:  50 │
                └─────┬─────┘
           ┌──────────┴──────────┐
           ▼                     ▼
      ┌─────────┐           ┌─────────┐
      │ 25 | 37 │           │ 75 | 87 │
      └─┬──┬──┬─┘           └─┬──┬──┬─┘
    ┌───┘  │  └───┐       ┌───┘  │  └───┐
    ▼      ▼      ▼       ▼      ▼      ▼
┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐
│10│20 │→│25│30 │→│37│45 │→│50│65 │→│75│80 │→│87│95 │
└──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘
    └──────────────────────────────────────────────→
                  Leaf linked list (→) enables range scans
```

8.2 LSM-Tree Compaction Flow

```
Writes arrive:
┌──────────────────────────┐
│ WAL (Write-Ahead Log)     │ ← sequential write, crash safety
└─────────────┬────────────┘
              ▼
┌──────────────────────────┐
│ MemTable (sorted, in-memory) │ ← fast writes, all keys sorted
└─────────────┬────────────┘
 (when full)  ▼
┌──────────────────────────────────────────┐
│ L0: [SST1][SST2][SST3][SST4]   ← may have overlaps │
└─────────────────┬────────────────────────┘
 (compaction)     ▼
┌──────────────────────────────────────────┐
│ L1: [SST_sorted_no_overlap ──────────────] │
└─────────────────┬────────────────────────┘
 (compaction)     ▼
┌──────────────────────────────────────────┐
│ L2: [SST_sorted_no_overlap — 10x larger ────────] │
└──────────────────────────────────────────┘

Read path: MemTable → L0 → L1 → L2 → ...
Bloom filters on each SSTable to skip misses quickly
```

8.3 Replication Topology

```
LEADER-FOLLOWER (Master-Slave):
  ┌──────────┐
  │  Leader  │◄── all writes
  │(Primary) │
  └────┬─────┘
       │ replication log (async or sync)
  ┌────┴──────────────┐
  ▼                   ▼
┌──────────┐     ┌──────────┐
│ Follower │     │ Follower │ ← reads can go here
│(Replica) │     │(Replica) │
└──────────┘     └──────────┘

MULTI-LEADER (Active-Active):
  ┌──────────┐   ┌──────────┐
  │  Leader  │◄─►│  Leader  │ ← writes accepted at both
  │ Region A │   │ Region B │
  └──────────┘   └──────────┘
    ↑ conflict resolution needed (last-write-wins, CRDT, custom)

LEADERLESS (Dynamo-style):
  ┌───────┐  ┌───────┐  ┌───────┐
  │ Node  │  │ Node  │  │ Node  │
  │  A    │  │  B    │  │  C    │
  └───────┘  └───────┘  └───────┘
  Write to W nodes; Read from R nodes; N total
  Quorum: W + R > N → consistent reads
  e.g., N=3, W=2, R=2: sloppy quorum
```

8.4 Sharding / Partitioning

```
RANGE PARTITIONING:
  Shard 1: user_id 1-1M          ┌──────────┐
  Shard 2: user_id 1M-2M         │ Router / │
  Shard 3: user_id 2M-3M ←───│ Proxy     │◄── Client writes
                                  └──────────┘

  Problem: Hot partitions if writes concentrate in range

HASH PARTITIONING:
  hash(user_id) % 3 = shard number
  Shard 0: hash % 3 == 0
  Shard 1: hash % 3 == 1
  Shard 2: hash % 3 == 2
  Advantage: Even distribution
  Disadvantage: Range queries hit all shards; resharding is expensive

CONSISTENT HASHING:
  Hash ring: 0 ────────────── 360
  Nodes placed at points on ring
  Key routes to nearest node clockwise
  Adding/removing node: only adjacent keys move → minimal resharding

  ┌──────────────── Ring ──────────────────┐
  │          Node A (90°)                  │
  │         /                              │
  │   Node D           Node B (180°)       │
  │  (270°)                                │
  │         \          /                   │
  │          Node C (0°)                   │
  └────────────────────────────────────────┘
```

9 NoSQL Deep Dive

9.1 When to Pick Which Database

Use Case	Pick	Why
User sessions, caching, leaderboards	**Redis (KV)**	Sub-ms latency, TTL, sorted sets
Product catalog, user profiles	**MongoDB (Document)**	Flexible schema, nested documents
Time-series IoT, metrics	**Cassandra (Column-Family)**	Write-optimized, time-ordered partition keys
Social graph traversal	**Neo4j (Graph)**	Relationship traversal is O(1) per hop
E-commerce orders	**PostgreSQL (SQL)**	ACID, complex queries, FK integrity
Analytics/reporting	**Redshift/BigQuery (OLAP)**	Columnar storage, massive parallel scan
Global low-latency KV	**DynamoDB**	Managed, auto-scale, single-digit ms

9.2 NoSQL Families Comparison

Family	Model	Query pattern	Scaling	Consistency	Examples
Key-Value	hash map	point lookups only	horizontal, trivial	eventual usually	Redis, DynamoDB, Memcached
Docume	JSON tree	field-level queries, secondary indexes	horizontal	configurable	MongoDB, Couchbase, Firestore
Column-Family	sparse table, columns grouped	partition key + sort key	horizontal, excellent	tunable (ONE/QUORUM/ALL)	Cassandra, HBase, Bigtable
Graph	nodes + edges	traversal, path finding	vertical (harder to shard)	ACID usually	Neo4j, Neptune, JanusGraph

9.3 Cassandra Architecture

```
Data model: Table → Partition Key → Clustering Keys → Columns

  Partition Key: determines which node(s) hold the data
  Clustering Key: determines ordering within a partition

CREATE TABLE events (
    user_id   UUID,        ← partition key → all user events on same node
    event_time TIMESTAMP,  ← clustering key → sorted within partition
    event_type TEXT,
    PRIMARY KEY (user_id, event_time)
) WITH CLUSTERING ORDER BY (event_time DESC);
```

Consistency levels: ONE, QUORUM, ALL, LOCAL_QUORUM

- QUORUM write + QUORUM read = strong consistency (W+R > N)
- ONE write + ONE read = fastest, but may read stale data

Tunable consistency: Application chooses per-operation. **Interview takeaway:** This is Cassandra's biggest feature — tune for your SLA.

9.4 DynamoDB

- **Primary key:** Partition key (required) + optional sort key
- **Access patterns drive design** — no ad-hoc queries; plan your indexes upfront
- **GSI (Global Secondary Index):** Different partition + sort key; async replication
- **LSI (Local Secondary Index):** Same partition key, different sort key; strongly consistent
- **Single-table design:** Multiple entity types in one table (over-loaded keys pattern)

```python
PK             | SK             | data
USER#alice     | PROFILE        | {name, email}
USER#alice     | ORDER#2024-01-01 | {items, total}
PRODUCT#widget | META           | {price, desc}
```

9.5 MongoDB

- Documents are BSON (binary JSON), up to 16MB
- **Sharding:** shard key determines distribution; avoid monotonically increasing keys
- **Replica sets:** Primary + N secondaries; automatic failover
- **Aggregation pipeline:** `$match → $group → $project → $sort → $limit`
- **No joins** (use `$lookup` but it's expensive); denormalize or use references

10 Real-World Examples

1. **Twitter timeline:** Fanout-on-write (pre-compute timelines) vs fanout-on-read. Redis sorted sets for timeline storage. Cassandra for tweet storage partitioned by user_id.
2. **Uber's trip matching:** PostgreSQL (PostGIS) for geospatial queries to find nearby drivers. Redis for real-time driver location updates.
3. **Amazon orders:** DynamoDB for order lookups by order_id (partition key). Aurora for complex reporting. Elasticache (Redis) for cart/session data.
4. **Netflix recommendations:** Cassandra for user watch history (write-heavy, time-ordered). Spark for batch processing. Elasticsearch for search.
5. **Banking transactions:** PostgreSQL or Oracle. Serializable isolation. Two-phase commit for distributed transactions. Strict ACID required — not negotiable.
6. **Facebook social graph:** TAO (custom graph KV store on top of MySQL). Read-heavy, eventually consistent, huge scale. Each edge/node cached in Memcached layer.

11 Real-Life Analogies

One great library — every concept is a shelf, a catalogue card, or a librarian's routine.

Concept	Analogy
Index	The card catalogue at the entrance — find a book without walking every shelf
B+Tree	The alphabetised catalogue drawers, with each leaf drawer linked to the next so range browsing is a single sweep across adjacent drawers
LSM-Tree	A returns cart that fills during the day (memtable) and is periodically merged and reshelved in strict order (compaction) — the cart is fast to drop books onto; the shelving run keeps the stacks sorted

Concept	Analogy
Clustered index	Books shelved in call-number order — the shelf layout IS the catalogue; finding call number 823.914 means walking straight to that spot, no detour
Non-clustered index	A subject-card in the catalogue that gives you a call number — you still have to walk the stacks to pull the physical book off the shelf
MVCC	Keeping the previous edition on the shelf so current readers aren't interrupted while the librarian quietly swaps in the new edition behind the desk
Transaction	Checking out a whole reading-list all-or-nothing — either every book is stamped and in your bag, or none leave the desk
ACID	Library rules: all books or none leave (atomic); the catalogue stays accurate after every checkout (consistent); two patrons can't grab the same last copy mid-checkout (isolated); the stamp record survives a power cut (durable)
Dirty read	Peeking at a book the librarian pulled but hasn't yet decided to add to the collection — it may go straight back to the supplier
Non-repeatable read	You note a book is on shelf 4, walk to the café, come back, and it's gone — another patron checked it out between your two glances
Phantom read	You count seven books on a topic, step away to photocopy one, return, and now there are eight — a new donation arrived in between
Deadlock	Patron A holds the atlas and is waiting for the dictionary; Patron B holds the dictionary and is waiting for the atlas — neither budges until the librarian intervenes and asks one to put their book down
Sharding	Splitting the collection across buildings by subject — Sciences in the East Wing, Humanities in the West Wing — so no single building is overwhelmed
Replication	Branch libraries holding copies of the most-requested titles — the main branch updates its master copy and the branches mirror it so local readers are never turned away
WAL	The librarian writes every incoming acquisition in the accession register before touching the shelves — if a shelf collapses mid-task, the register lets her reconstruct exactly where each book belongs
Pessimistic locking	Clipping a "Being Used — Do Not Remove" tag on a book the moment a patron sits down with it, so no one else can grab it from under them
Optimistic locking	No tag on the book while reading; at the checkout desk the librarian checks whether the edition number on the stamp card still matches — if someone else already updated the record, she hands the book back and asks the patron to start over
Normalization	One master author-card per person in the catalogue — every book's record points to it rather than repeating the author's biography on every card
Denormalization	Photocopying the author biography onto every book's catalogue card — faster to read in one place, but when the author changes address every card must be updated individually
Joins	Cross-referencing the authors' catalogue with the books' catalogue — match each author card to every book card that shares the same author code to build the full picture
NoSQL	A flexible scrapbook section where patrons paste in newspaper clippings, maps, and sticky notes — no rigid card format required, but you cannot run the same catalogue queries you would on a bound volume

12 Memory Tricks / Mnemonics

12.1 ACID

"A Consistent Isolation Delivers"

- **A**tomicity — All or nothing
- **C**onsistency — Valid state to valid state
- **I**solation — Concurrent txs don't interfere
- **D**urability — Committed = permanent

12.2 Isolation Levels (weakest to strongest)

"Read Uncommitted Criminals Repeatedly Steal" → RU, RC, RR, S

1. **R**ead **U**ncommitted — see dirty data
2. **R**ead **C**ommitted — no dirty reads
3. **R**epeatable **R**ead — no non-repeatable reads
4. **S**erializable — full isolation

   Anomaly prevention by level (cumulative):

- RC prevents: **D**irty reads (mnemonic: **C**ommitted stops **D**irt)
- RR adds: no **N**on-repeatable reads (mnemonic: **R**epeatable stops **N**ot-same)
- S adds: no **P**hantoms (mnemonic: **S**erial stops **P**hantoms)

12.3 Joins Memory Aid

"INNER only matches, LEFT keeps left, RIGHT keeps right, FULL keeps all"

- Think of Venn diagrams: INNER = intersection, LEFT = left circle, RIGHT = right circle, FULL = union

12.4 B+Tree vs LSM Choice

"B+Tree Reads, LSM Writes"

- B+Tree: **B**etter for **R**eads (random lookups, range scans)
- LSM: **L**oves **S**equential **M**assive writes

12.5 Window Function Rank Types

"RANK has Gaps, DENSE doesn't, ROW always unique"

- 1, 2, 2, **4** → RANK (gap after tie)
- 1, 2, 2, **3** → DENSE_RANK (no gap)
- 1, 2, 3, **4** → ROW_NUMBER (always distinct)

12.6 NoSQL Family by Access Pattern

"KV=Speed, Doc=Flexible, Column=Scale, Graph=Traverse"

12.7 CAP Theorem

"Consistent systems Pause during partitions; Available systems Accept stale data"

13 Common Interview Questions

13.1 SQL Query Questions (Frequently Asked)

SQL

```sql
-- 1. Second highest salary
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
-- Or: SELECT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- 2. Nth highest salary (general)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked WHERE rnk = N;

-- 3. Find duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- 4. Delete duplicates, keep lowest id
DELETE FROM users WHERE id NOT IN (
    SELECT MIN(id) FROM users GROUP BY email
);

-- 5. Employees earning more than their manager
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- 6. Running total of sales
SELECT date, amount,
       SUM(amount) OVER (ORDER BY date) AS running_total
FROM sales;

-- 7. Consecutive logins (gaps and islands)
WITH login_data AS (
    SELECT user_id, login_date,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)
           - DENSE_RANK() OVER (PARTITION BY user_id ORDER BY login_date) AS grp
    FROM logins
)
SELECT user_id, MIN(login_date), MAX(login_date), COUNT(*) AS streak_len
FROM login_data
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;

-- 8. Pivot / crosstab (using CASE)
SELECT
    student_id,
    SUM(CASE WHEN subject = 'Math' THEN score END) AS math,
    SUM(CASE WHEN subject = 'Science' THEN score END) AS science
FROM grades
GROUP BY student_id;
```

13.2 System-Level DB Questions

Q: How does PostgreSQL handle a SELECT query internally?

A:

1. **Parser** — tokenize SQL → parse tree
2. **Rewriter** — apply view definitions, rules
3. **Planner/Optimizer** — generate query plan (consider indexes, join orders, cost estimates from statistics)

4. **Executor** — execute plan node by node, return rows

Q: Explain how an UPDATE causes more work than it looks.

- Read original row (may involve index scan)
- Acquire X lock
- Write new version (MVCC in Postgres: new tuple, old marked deleted)
- Update all secondary indexes pointing to this row
- Write to WAL
- Release lock at commit

Q: What happens when two transactions update the same row simultaneously?

- First writer acquires X lock
- Second writer blocks waiting for X lock
- When first commits/rolls back → second proceeds
- If both started at same time (deadlock scenario) → DB detects and kills one

Q: When would you add an index? When would you not?
Add index when:

- Column appears in WHERE/JOIN/ORDER BY frequently
- Column has high selectivity (many distinct values)
- Query response time is too slow and plan shows Seq Scan on large table

Don't add index when:

- Table is small (full scan is fine)
- Column has low selectivity (gender, boolean)
- Table is write-heavy (indexes slow every INSERT/UPDATE/DELETE)
- Column is rarely queried

Q: Explain consistency models in distributed databases.

- **Strong consistency:** Read always sees latest write (Spanner, ZooKeeper)
- **Eventual consistency:** Reads converge to latest write eventually (Cassandra, DynamoDB default)
- **Read-your-own-writes:** You always see your own writes (common requirement)
- **Monotonic reads:** Won't see older state after seeing newer state
- **Causal consistency:** Operations with causal dependencies are ordered

13.3 Follow-Up Questions Interviewers Love

- "Your B+Tree index is on (a, b). Can it speed up `WHERE b = 5`?" → No — leftmost prefix rule. Index on b separately needed.
- "You have an index but the query planner doesn't use it. Why?" → Low selectivity, stale stats, small table, OR condition, leading wildcard LIKE '%foo'.
- "MVCC prevents locking for reads. Is there any case readers still block?" → DDL operations (ALTER TABLE), lock table, schema-level changes.
- "How does Cassandra handle a write if one replica is down?" → Hinted handoff: write stored at coordinator with hint, replayed when node recovers.

14 Senior-Level Discussion Points

14.1 Index Trade-offs

Write amplification with indexes:

- Every secondary index must be updated on every INSERT/UPDATE/DELETE
- Table with 5 secondary indexes: 1 logical write → potentially 6 physical writes
- **Mitigation:** Audit indexes, drop unused ones. Use pg_stat_user_indexes to find unused indexes.

Index bloat:

- B+Trees have unused space due to page splits and deletions
- PostgreSQL: `REINDEX` to rebuild, `VACUUM` to reclaim
- Monitor with pgstattuple extension

Partial indexes:

```sql
-- Only index active users — much smaller, faster
CREATE INDEX idx_active_users ON users(email) WHERE active = true;
```

Expression indexes:

```sql
-- Enable case-insensitive search efficiently
CREATE INDEX idx_lower_email ON users(LOWER(email));
-- Query must use LOWER(email) = '...' to use this index
```

14.2 Write vs Read Optimization

Optimize For	Strategies
Read	More indexes, denormalization, materialized views, caching, read replicas
Write	Fewer indexes, normalization, batch writes, async writes, LSM-tree storage
Both	CQRS (Command Query Responsibility Segregation) — separate write/read models

14.3 Hot Partitions

Problem: If partition key is not chosen carefully, one partition gets all traffic.

```
Bad: partition by creation_date (all today's traffic on one node)
Bad: partition by status (all "active" users on one node)
Good: partition by user_id hash (even distribution)
```

Mitigation for hot partitions:

1. Add random suffix to key (write spread): `user_id + "_" + random(0,9)`
2. Read by trying all suffixes and combining results
3. Use a separate cache layer for hot keys
4. Adaptive/compound partition keys

14.4 Replication Lag

Problem: Leader accepts write; follower replication is async; brief window of stale data on follower.

Strategies:

- Read from leader after a write (read-your-own-writes consistency)
- Monotonic reads: user always routed to same replica
- Track replication position; application waits if read replica is too far behind
- Semi-synchronous replication: at least 1 replica must ack before commit

14.5 Query Optimization Workflow

1. **Identify slow queries** — `pg_stat_statements`, slow query log
2. **EXPLAIN ANALYZE** — look for Seq Scan on large tables, high row estimates
3. **Check statistics** — `ANALYZE table` to update planner stats
4. **Add missing index** — check if selectivity warrants it
5. **Rewrite query** — avoid OR (use UNION), avoid SELECT *, avoid functions on indexed columns in WHERE
6. **Consider schema changes** — normalization, adding denormalized fields, partitioning
7. **Hardware** — more RAM for buffer cache, faster disk, read replicas

15 Typical Mistakes Candidates Make

1. **Confusing isolation levels with lock types** — isolation levels are *behaviors*; locks are the *mechanism*. MVCC achieves isolation without most locks.
2. **Saying "just add an index"** — without explaining the selectivity check, the write overhead cost, or that the planner might not use it.
3. **Claiming Cassandra is strongly consistent** — it's AP by default; tunable but defaults to eventual. Know the default.
4. **Forgetting phantom reads in Repeatable Read** — standard SQL allows phantoms at RR; only Serializable prevents them. (Note: InnoDB extends RR with gap locks.)
5. **Misunderstanding MVCC** — "No locking" is oversimplified. MVCC eliminates reader-writer blocking, not writer-writer conflicts. Writers still need locks.
6. **Using UUIDs as clustered index in InnoDB** — causes random page splits, fragmentation, terrible write performance. Use auto-increment or time-ordered UUIDs (UUIDv7).
7. **Confusing horizontal partitioning (sharding) with vertical partitioning** — sharding = split rows across nodes; vertical = split columns across tables/nodes.
8. **Ignoring the "N+1 query problem"** — SELECT 10 users, then SELECT orders for each user = 11 queries. Use JOINs or eager loading.
9. **Assuming CAP means "pick two of three always"** — you always need partition tolerance in distributed systems. Real choice is C vs A during partition.
10. **Forgetting VACUUM in PostgreSQL** — MVCC creates dead tuples; without vacuum, table bloat kills performance.

16 How This Connects To Other Topics

Topic	Connection
Distributed Systems	Replication, consensus (Paxos/Raft for leader election), CAP theorem, distributed transactions (2PC/SAGA)
System Design	Database selection (SQL vs NoSQL), sharding strategy, caching layer (Redis), read replicas, data modeling
Operating Systems	Page cache, fsync, memory-mapped files (mmap), file descriptors, I/O scheduling
Computer Networks	Client-server protocol (MySQL protocol), connection pooling, TCP for reliability, replication over network
Performance Engineering	Index tuning, query optimization, connection pool sizing, buffer pool tuning, benchmark tools (pgbench, sysbench)
Concurrency	Locks, MVCC, deadlock detection algorithms, condition variables for lock waits
Data Structures	B+Tree, skip list (Redis sorted sets), hash tables (hash indexes), LSM-tree
Storage	Sequential vs random I/O, SSD vs HDD impact on LSM vs B+Tree, WAL, page layout

17 FAANG Interview Tips

1. **Always clarify the access pattern first.** "What queries will this serve?" determines everything — schema, index, DB choice.
2. **Know the trade-off triangle:** Performance vs Consistency vs Availability. State your choice and justify.
3. **For SQL questions:** Think about edge cases — NULLs in JOIN conditions, duplicates, ties in ranking. Mention them proactively.
4. **For system design:** Don't just say "I'll use DynamoDB." Explain: partition key choice, GSI needs, consistency requirements, estimated RPS, TTL usage.
5. **Mention EXPLAIN:** When discussing optimization, always mention "I would run EXPLAIN ANALYZE to see the query plan." Shows you know the workflow.
6. **Index questions:** Cover creation, selectivity, covering indexes, composite index column order (leftmost prefix rule).
7. **Cassandra questions:** Explain data modeling around access patterns, partition key importance, and consistency level tuning. Amazon/Netflix questions will go deep here.
8. **Google-specific:** Know Spanner's TrueTime (bounded clock uncertainty $\rightarrow$ external consistency). Know Bigtable's architecture (column families, SSTable-based).
9. **Deadlock questions:** Draw the wait-for graph. Explain victim selection. State that application should retry on deadlock.
10. **Always mention trade-offs.** "We can improve read latency by adding an index, but that increases write latency and storage. Given this is read-heavy workload at 95:5 ratio, the trade-off is worth it."

18 Revision Cheat Sheet

18.1 10-Minute Summary

SQL: Relational model + ACID + schema-on-write. Joins combine tables. CTEs name subqueries. Window functions compute over rows without collapsing.

Indexing: B+Tree for most databases (all data in leaves, linked list for ranges). LSM-tree for write-heavy (sequential writes, periodic compaction). Clustered = data stored in index order. Secondary = separate structure + pointer to row.

ACID & Isolation: Atomicity+Durability via WAL. Isolation via MVCC (readers don't block writers). Four levels: Read Uncommitted → Read Committed → Repeatable Read → Serializable. Each level prevents additional anomalies.

Locking: Pessimistic = lock upfront. Optimistic = check version at commit. Deadlock = circular wait → DB kills one victim.

Distributed: Replication (leader-follower, multi-leader, leaderless). Sharding (range, hash, consistent hashing). CAP theorem: partition tolerance always needed → choose C or A during partition.

NoSQL: KV=speed, Document=flexibility, Column-family=write-scale, Graph=traversal. Choose based on access pattern.

18.2 Key Numbers to Know

- B+Tree height: 3-4 levels for millions of rows (branching factor ~200)
- Cassandra recommended partition size: < 100MB, < 100K rows
- DynamoDB item size limit: 400KB
- MongoDB document size limit: 16MB
- Typical replication lag (async): milliseconds to seconds
- Index seek cost: O(log n); full scan: O(n)

18.3 Most Important Concepts for Interviews

Rank	Concept	Why Important
1	Isolation Level × Anomaly Matrix	Asked in almost every DB interview
2	B+Tree structure and why it's used	Fundamental to understanding indexes
3	MVCC	Explains PostgreSQL/MySQL behavior
4	Window functions (RANK, ROW_NUMBER, LAG)	Common SQL coding questions
5	Clustered vs secondary index	Critical for performance questions
6	LSM-Tree vs B+Tree trade-off	Explains Cassandra/RocksDB choices
7	Sharding strategies	Core system design component
8	EXPLAIN / query plan reading	Shows practical DB experience
9	CAP applied to specific DBs	System design discussion
10	When to use SQL vs which NoSQL	Architecture decision questions

18.4 Cheat-Sheet Summary Table

Question Type	Quick Answer
Prevent dirty reads?	Read Committed or above
Prevent phantom reads?	Serializable (or InnoDB RR with gap locks)
Readers block writers?	No in MVCC (PostgreSQL, MySQL InnoDB)
Range scan data structure?	B+Tree (leaf linked list)
Write-heavy storage?	LSM-Tree (Cassandra, RocksDB)
Delete + keep lowest ID?	DELETE WHERE id NOT IN (SELECT MIN(id) ... GROUP BY ...)
Top N per group?	ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)
Even shard distribution?	Hash partitioning

Question Type	Quick Answer
Resharding with minimal data movement?	Consistent hashing
Global ACID + distributed?	Google Spanner, CockroachDB
Session/cache store?	Redis
Time-ordered events at scale?	Cassandra with time-based clustering key
Social graph traversal?	Graph DB (Neo4j, Neptune)
Deadlock resolution?	DB kills victim; application retries
Index not used by planner?	Check selectivity, stale stats, OR conditions, functions on column